

LAB ASSIGNMENT – 5

**CARTPOLE BALANCING USING
DEEP Q-NETWORK (DQN)**

Reinforcement Learning

Submitted by

SANJU BHASKAR

BL.EN.R4AIE26007

Python Implementation using Gymnasium and PyTorch

1. Title

Implementation of CartPole Balancing using a Deep Q-Network (DQN).

2. Objective

The objective of this experiment is to formulate the CartPole balancing problem as a reinforcement learning problem and train an agent using a Deep Q-Network (DQN). The agent learns to select left or right actions so that the pole remains balanced for as many time steps as possible. The experiment records training rewards, generates a Reward-versus-Episodes learning curve, evaluates the trained agent, and analyzes the difference between the best model checkpoint and the final training checkpoint.

3. Application Description

CartPole is a control problem in which a pole is attached to a moving cart. At every time step, the reinforcement learning agent applies a force to move the cart to the left or right. The objective is to keep the pole upright while keeping the cart within the allowed operating region. An episode ends when the environment reaches a terminal or truncated condition.

The experiment used the Gymnasium CartPole-v1 environment. The state is represented by four continuous variables and the action space contains two discrete actions.

4. Reinforcement Learning Formulation

RL Component	CartPole Formulation
Agent	DQN-based reinforcement learning agent
Environment	Gymnasium CartPole-v1
State	Cart position, cart velocity, pole angle, pole angular velocity
State dimension	4
Actions	0 = Move Left; 1 = Move Right
Action dimension	2
Reward	Reward obtained for maintaining the pole/cart system within the allowed conditions
Episode	One complete interaction sequence until termination or truncation
Terminal condition	Environment reports terminated=True or truncated=True
Goal	Maximize cumulative reward by maintaining balance for more time steps

4.1 State Representation

The state supplied to the DQN is a four-dimensional vector:

$$\mathbf{S} = [\mathbf{x}, \dot{\mathbf{x}}, \theta, \dot{\theta}]$$

- x – cart position.
- $\dot{x}$ – cart velocity.
- θ – pole angle.
- $\dot{\theta}$ – pole angular velocity.

4.2 Action Representation

The environment provides two discrete actions. Action 0 moves the cart to the left and action 1 moves the cart to the right.

5. Algorithm Selected – Deep Q-Network

DQN was selected because the CartPole state variables are continuous. Instead of maintaining a large tabular representation of every possible state, a neural network approximates the action-value function $Q(s,a)$. For a given state, the network produces two Q-values, one for each available action.

5.1 DQN Architecture

Layer	Configuration
Input layer	4 neurons – CartPole state
Hidden layer 1	128 neurons + ReLU
Hidden layer 2	128 neurons + ReLU
Output layer	2 neurons – $Q(\text{left})$, $Q(\text{right})$

The implemented DQN also uses an online/policy network and a target network. The target network is periodically synchronized with the policy network to provide more stable targets during learning.

5.2 DQN Target Equation

$$y = r + \gamma \max_{a'} Q(s', a')$$

For a terminal transition, the future-value term is omitted, so the target reduces to the immediate reward. The network minimizes the difference between the predicted Q-value for the selected action and the target value. The implementation uses Smooth L1 (Huber) loss.

5.3 Epsilon-Greedy Exploration

The agent uses epsilon-greedy action selection. With probability ϵ , a random action is selected for exploration; otherwise, the action with the highest predicted Q-value is selected. In this experiment ϵ starts at 1.0 and gradually decays toward 0.05.

5.4 Experience Replay

Each interaction is stored as (state, action, reward, next state, done). Random mini-batches are sampled from replay memory to train the network. This reduces the dependence between consecutive experiences and supports more stable learning.

6. Implementation and Training Setup

Parameter	Value
Programming language	Python
Environment	Gymnasium CartPole-v1
Deep learning framework	PyTorch
Device	CPU
Training episodes	500
Learning rate	0.001
Discount factor (γ)	0.99
Batch size	64
Replay memory	50,000
Minimum replay size before training	1,000
Initial epsilon	1.0
Minimum epsilon	0.05
Epsilon decay	0.995
Target network update frequency	Every 10 episodes
Hidden layer size	128, 128
Random seed	42

6.1 Training Algorithm

1. Reset the CartPole environment and obtain the initial state.
2. Select an action using the epsilon-greedy strategy.
3. Execute the action in the environment.
4. Receive the reward, next state, and terminal/truncated status.
5. Store the transition in the replay buffer.
6. Sample a random mini-batch once sufficient experiences are available.
7. Calculate the current Q-values and DQN target values.
8. Compute the loss and update the policy network using backpropagation.
9. Periodically copy the policy network parameters to the target network.
10. Reduce epsilon after each episode and record the total episode reward.
11. Calculate the 20-episode moving average and save a checkpoint whenever a new best average is obtained.

7. Experiment 1 – Initial DQN Training

The first run trained the DQN for 500 episodes and evaluated the final model saved after episode 500. The purpose of this run was to establish the baseline training behavior.

Measure	Experiment 1 Result
Training episodes	500
Average training reward	87.64
Maximum training reward	500
Minimum training reward	10
Final 20-episode average	87.55
Evaluation episodes	20
Average evaluation reward	91.70
Maximum evaluation reward	101
Minimum evaluation reward	77
Successful episodes (threshold ≥ 195)	0/20
Success rate	0%
Final demonstration reward	96

The baseline learning curve showed that the moving average increased substantially during the middle of training but declined again toward episode 500. This observation motivated the best-model checkpointing strategy used in Experiment 2.

8. Experiment 2 – Best-Model Checkpointing

In Experiment 2, the DQN architecture and training settings were kept the same as the initial experiment. The change was that the model was saved whenever the 20-episode moving average reached a new maximum. Both the best checkpoint and the final checkpoint were then evaluated over 20 episodes.

8.1 Learning Curve

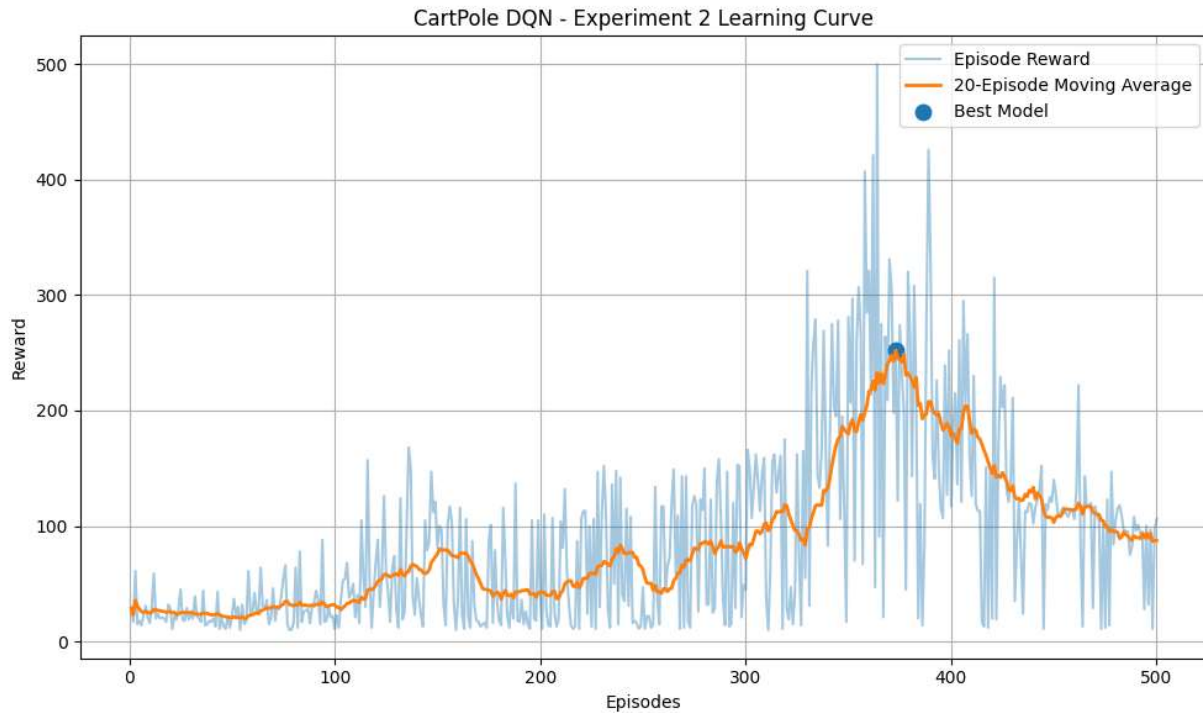


Figure 1. CartPole DQN Experiment 2 learning curve. The blue curve represents episode reward, the orange curve represents the 20-episode moving average, and the marked point identifies the best model checkpoint.

8.2 Training Results

Measure	Experiment 2 Result
Training episodes	500
Average training reward	87.64
Maximum training reward	500
Minimum training reward	10
Best 20-episode moving average	252.30
Best model episode	373
Final 20-episode moving average	87.55

8.3 Best Model vs Final Model

Evaluation Measure	Best Model	Final Model
Checkpoint	Episode 373	Episode 500
Average evaluation reward	183.85	94.80
Maximum evaluation reward	196	103
Minimum evaluation reward	174	89
Successful episodes	1/20	0/20
Success rate	5.00%	0.00%

For this experiment, an evaluation reward of 195 or greater was defined as a successful episode. Under this selected criterion, the best model produced 1 successful episode out of 20, while the final model produced none.

9. Performance Analysis

The training curve indicates three broad stages. During the early episodes, rewards remained relatively low while the agent explored the environment. During the middle part of training, the 20-episode moving average

increased substantially, reaching 252.30 at episode 373. Individual episode rewards also reached the maximum observed value of 500. After the best checkpoint, however, the moving average decreased and ended at approximately 87.55 during the final 20 episodes.

The evaluation results reflect this difference. The best checkpoint obtained at episode 373 achieved an average evaluation reward of 183.85 over 20 episodes, while the final checkpoint at episode 500 achieved 94.80. The difference between these averages was 89.05 reward points. Thus, for this particular run, the checkpoint with the best training moving average also produced the higher evaluation average.

The result should be interpreted as an observation from this experimental run rather than a general guarantee about DQN. Continued neural-network updates can change the learned Q-function, and reinforcement learning performance can fluctuate between training checkpoints.

10. Observations

12. The CartPole problem was successfully formulated using a four-dimensional continuous state and two discrete actions.
13. The DQN agent initially obtained low rewards while exploration was high.
14. Training produced substantial improvement during the middle portion of the 500-episode run.
15. The highest observed individual training reward was 500.
16. The highest 20-episode moving average was 252.30 at episode 373.
17. Performance decreased after the best checkpoint and the final 20-episode moving average was 87.55.
18. The best checkpoint achieved an average evaluation reward of 183.85 over 20 episodes.
19. The final checkpoint achieved an average evaluation reward of 94.80.
20. Using the selected success threshold of 195, the best checkpoint achieved a 5% success rate and the final checkpoint achieved 0%.
21. The experiment demonstrates the usefulness of monitoring training checkpoints rather than assuming that the last training model is the best-performing model.

11. Final Demonstration

The best checkpoint was loaded and used in a rendered CartPole environment for the final demonstration. The observed demonstration episode produced a reward of 190. This result is close to the selected evaluation threshold of 195 and is consistent with the evaluation results showing that the best model learned substantially longer balancing behavior than the final checkpoint.

12. Conclusion

A Deep Q-Network based reinforcement learning agent was implemented for the CartPole balancing problem using Python, Gymnasium, and PyTorch. The state consisted of cart position, cart velocity, pole angle, and pole angular velocity, while the action space contained left and right movements. The DQN used a neural network to approximate Q-values, together with epsilon-greedy exploration, experience replay, and a target network.

The agent was trained for 500 episodes. The best 20-episode moving average was 252.30 at episode 373. Evaluation of this checkpoint produced an average reward of 183.85, compared with 94.80 for the final episode-500 checkpoint. The results show that the agent learned useful CartPole balancing behavior, while

also demonstrating that performance can vary during training and that the final checkpoint is not necessarily the highest-performing checkpoint observed in a particular run.

16. References

Sutton, R. S., and Barto, A. G., Reinforcement Learning: An Introduction, 2nd Edition.